

Term Project – Proposal

Vulnerability Tracking System

Dozent: Stephan Bökelmann sboekelmann@ep1.rub.de Technische Hochschule Georg Agricola, Bochum

Author: Meriem Mornagui

Module: Introduction to Database Management Systems · THGA Bochum

Submitted to: Stephan Bökelmann · sboekelmann@ep1.rub.de

Planned effort: approximately 40 hours over no more than two months

1 Application domain and motivation

Large organisations manage many IT assets across on-premises and cloud environments. They must know which assets are affected by known vulnerabilities (CVEs), which team owns each asset, who is responsible for remediation, and whether remediation deadlines are being met. Spreadsheets quickly become difficult to maintain because they cannot reliably enforce relationships, validate status values, prevent duplicate findings, or answer analytical questions without manual work.

The proposed application replaces this spreadsheet-based process with a relational database, a REST API, and a desktop frontend. It manages teams, IT assets, known vulnerabilities, and findings that connect affected assets to vulnerabilities. Each finding records its status, discovery date, remediation deadline, and optional resolution date.

The project is inspired by real-world vulnerability-management workflows, but it is an independent educational prototype. Only fictional teams, assets, addresses, vulnerabilities, dates, and credentials will be used. No internal company data, code, screenshots, or documentation will be included.

Usage scenario 1. A security analyst opens the frontend and filters the findings for severity *Critical* and status *Open*. The analyst sees the affected asset and its responsible team and can create or resolve findings through the API.

Usage scenario 2. A team lead requests all overdue findings belonging to their team. The system returns the affected assets, CVE identifiers, severity levels, and past-due remediation dates so that the team can prioritise its work.

2 Data model – ER diagram

Figure 1 shows four entity types. A team owns one or more assets. An asset represents a physical or virtual IT resource such as a server, laptop, or cloud instance. A vulnerability represents a known security weakness identified by a unique CVE identifier and a severity level. A finding links one asset to one vulnerability and stores remediation-specific information.

One asset may be affected by many vulnerabilities, and one vulnerability may affect many assets. Therefore, asset and vulnerability have an N:M relationship. The associative entity **finding** resolves this relationship into two 1:N relationships.

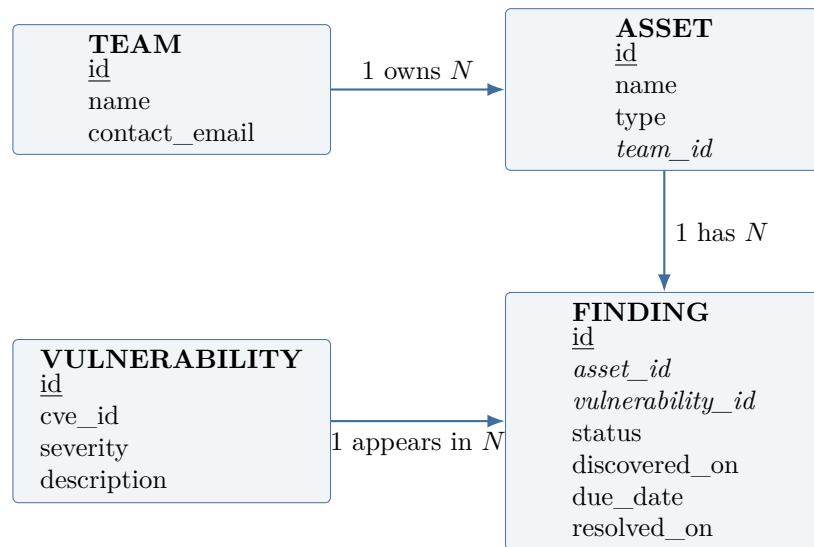


Abbildung 1: ER diagram of the Vulnerability Tracking System, including entities, key attributes, foreign keys, relationships, and cardinalities. The N:M relationship between **asset** and **vulnerability** is resolved by **finding**.

3 From model to relational schema

The relational schema is derived directly from the ER diagram. Primary keys are underlined and foreign keys are written in italics.

- **team**(id, name, contact_email)
- **asset**(id, name, type, *team_id*)
- **vulnerability**(id, cve_id, severity, description)
- **finding**(id, *asset_id*, *vulnerability_id*, status, discovered_on, due_date, resolved_on)

The database uses the following integrity constraints:

- **team.name** and **vulnerability.cve_id** are unique.
- The combination (**asset_id**, **vulnerability_id**) is unique, so the same vulnerability cannot be added twice to the same asset.
- **severity** is restricted to *Critical*, *High*, *Medium*, or *Low* by a **CHECK** constraint.
- **status** is restricted to *Open*, *In Progress*, or *Resolved* by a **CHECK** constraint.
- **due_date** must not be earlier than **discovered_on**.
- **resolved_on** must be present only when the status is *Resolved*.

Normalisation. The schema is in third normal form (3NF). Each table represents one subject, every non-key attribute depends on its table's primary key, and no non-key attribute depends transitively on another non-key attribute. Team information is stored once in **team**; asset records reference it through **team_id**. Vulnerability descriptions and severity levels are stored once in **vulnerability**, while **finding** stores only relationship-specific remediation data.

4 API design

The REST API is implemented with FastAPI. Read operations do not require an API key. Write operations require a valid `X-API-Key` header. Request bodies and responses are validated with Pydantic models, while SQL queries use parameters rather than string concatenation.

Method	Path	Purpose
GET	<code>/assets</code>	List assets with team name and number of open findings (JOIN + aggregation).
GET	<code>/findings</code>	List findings with asset, team, CVE, severity, status, and dates (multiple JOINS); optional filters.
GET	<code>/findings/{id}</code>	Return the complete details of one finding.
POST	<code>/findings</code>	Create a finding after validating foreign keys and duplicate combinations; requires <code>X-API-Key</code> .
POST	<code>/findings/{id}/resolve</code>	Change an open finding to <i>Resolved</i> and set the resolution date; requires <code>X-API-Key</code> .

For example, `GET /findings?status=Open&severity=Critical` returns only open critical findings. The endpoint `GET /assets` joins `asset` and `team`, left-joins `finding`, and groups the result by asset to calculate the number of open findings.

5 Frontend prototype

The frontend is a Python/tkinter desktop application. It communicates only with the REST API and never connects directly to PostgreSQL. At startup, an initial connection dialog requests the API URL and API key. The key is kept in memory for the current session and is not written to disk. The main window provides an asset overview and access to the finding list, creation form, and resolution action.

Figures 2–5 show the planned user interface. Figure 2 collects the connection information. Figure 3 displays the asset overview and aggregated number of open findings. Figure 4 provides the validated input fields needed to create a finding. Figure 5 shows the confirmation step used to resolve an existing finding.

The application will be packaged as a Debian `.deb` installer. The Python application will first be bundled with PyInstaller and then packaged with `fpm`.

6 Operation with Docker Compose

The backend consists of two Docker containers orchestrated with Docker Compose. The `postgres` service creates the schema and seed data from an `init.sql` script on first startup. The `api` service is built from the `api/` directory, waits for a healthy database connection, and exposes FastAPI on port 8000.

Database credentials and the API key are read from a `.env` file that is excluded from Git. The repository contains a `.env.example` file with safe placeholder values so that the required variables are documented without publishing secrets.

Deployment on the lecture server consists of the following steps:

1. Copy `.env.example` to `.env` and set secure values.
2. Run `docker compose up -d --build` to start PostgreSQL and the API.

Connect to Vulnerability Tracker

API URL

X-API-Key

Abbildung 2: Initial connection dialog for the API URL and X-API-Key.

Vulnerability Tracker – Assets

Assets Findings New Finding

Asset	Type	Team	Open
web-01	Server	Platform	3
laptop-17	Laptop	Finance	1
aws-prod	Cloud	Platform	5

Abbildung 3: Main asset screen populated by GET /assets.

Create New Finding

Asset

Vulnerability

Status

Discovered

Due date

Abbildung 4: Finding creation form using POST /findings.

Resolve Finding

Finding #42

Asset: web-01

CVE: CVE-2026-1234

Severity: Critical

Current status: Open

Resolution

Abbildung 5: Resolution screen using POST /findings/{id}/resolve.

3. Verify the services with `docker compose ps` and the API health endpoint.
4. Install the frontend with `sudo dpkg -i vuln-tracker_0.1.0_amd64.deb`.
5. Launch `vuln-tracker` and enter the API URL and API key.

7 Scope, milestones and risks

Scope. The minimum viable product contains four tables, five REST endpoints, one tkinter desktop application with four principal screens, Docker Compose deployment, automated tests, documentation, and a short demonstration video. User accounts, external CVE feeds, e-mail notifications, and historical status auditing are explicitly outside the project scope.

Milestones and schedule.

Week	Milestone	Est. hours
1	Final ER model, SQL schema, constraints, seed data, and local Docker Compose setup	8
2–3	FastAPI models and all endpoints, authentication guard, JOIN and aggregation queries	12
4–5	tkinter frontend, API integration, validation, and Debian packaging	12
6	Automated tests, deployment verification, documentation, and video	8

Risks and mitigation.

- **Packaging risk:** PyInstaller or `fpm` may miss runtime dependencies. Packaging will therefore be tested early in a clean Debian-based environment rather than left until the

final week.

- **Deployment risk:** The API may start before PostgreSQL is ready. Docker health checks and service dependencies will be used.
- **Query-performance risk:** Filtering and aggregation may become slow as the number of findings grows. Indexes will be created on `finding.asset_id`, `finding.vulnerability_id`, `finding.status`, and `finding.due_date`.
- **Scope risk:** Optional features will only be attempted after all acceptance criteria for the minimum viable product are satisfied.

8 Acceptance criteria and testing

Acceptance criteria.

- The database starts from a clean Docker volume and creates all tables, constraints, indexes, and seed records without manual SQL commands.
- A finding referencing a non-existent asset or vulnerability is rejected with a documented client-error response.
- A duplicate combination of asset and vulnerability is rejected.
- `GET /assets` returns the correct number of open findings for every asset, including assets with zero open findings.
- `GET /findings` correctly applies status and severity filters.
- Write endpoints return HTTP 401 when the API key is missing or invalid.
- Resolving a finding changes its status to *Resolved* and records the resolution date without changing it on a second resolve attempt.
- The frontend displays API errors clearly and does not crash when the API is unreachable or validation fails.

Test plan.

- **Database tests:** verify primary-key uniqueness, foreign-key constraints, unique constraints, CHECK constraints, and date rules against a disposable test database.
- **API tests:** use `pytest` and FastAPI's `TestClient` for successful requests and error cases for all five endpoints.
- **Query tests:** compare JOIN, filter, and aggregation results against known seed data, including an asset with no findings.
- **Authentication tests:** test missing, incorrect, and correct API-key headers on both write endpoints.
- **Frontend tests:** manually test connection failure, successful loading, form validation, finding creation, and resolution.
- **Deployment test:** install the generated `.deb` package in a clean environment and verify communication with the Dockerised API.

Effort budget: The proposed minimum viable product, documentation, and video are

designed to fit within approximately **40 hours of work** over at most **two months**.